

Technical Report

How the backend, the ML forecast service and RAG work together

Project	Suitability-aware payoff simulator for structured products (ELN, DCD, CPN)
Requirements baseline	PRD.md v2.0 (2026-10-03)
Report date	4 October 2026
Scope	Backend (apps/api), the Mode A forecast service (GARCH Monte Carlo), and the RAG explanation and chat layer. Frontend and database are covered only where they touch these.
Reading guide	Sections marked Built exist in the repository and are tested. Agreed means fixed in the PRD or contract documents. Proposed is a recommended design that still needs your decision.

Honest status. Today the repository contains the PRD, the forecast contract, the shared Zod schemas, the backend forecast client, the Mode A scenario helpers and 30 unit tests. The Express server, the payoff, risk and suitability engines, market data, persistence and the whole RAG service are still to be built. The ML forecast service is being built separately. This report explains how all of it is meant to work and marks which parts exist.

Contents

1. Summary	4
2. System architecture	5
3. Backend	6
3.1 Layers (Agreed)	6
3.2 Endpoints (Agreed names, schemas pending)	6
3.3 Request lifecycle	6
3.4 Validation (Built: tenor and forecast rules; Proposed: product terms)	7
3.5 Payoff engines (Proposed code, formulas Agreed)	7
3.6 Mode A scenario processing (Built)	8
3.7 Risk engine (Proposed)	8
3.8 Suitability engine (Rules Agreed as examples; mapping Proposed)	8
3.9 Persistence and failover (Agreed architecture)	9
3.10 Errors and security	9
4. ML forecast service (Mode A)	9
4.1 What it predicts, and what it does not	9
4.2 Data and returns	9
4.3 The GARCH(1,1) model	10
4.4 Monte Carlo simulation	10
4.5 Cases, fan and samples	11
4.6 Drift (average return)	11
4.7 Training window	11
4.8 Backtest: how forecast error is measured	12
4.9 Contract with the backend (Agreed)	12
4.10 Optional ML variant	12
4.11 Limitations to state in the product	12
5. RAG: retrieval-augmented explanation and chat	13
5.1 Role	13
5.2 What the explanation must cover (PRD §7.3)	13
5.3 Corpus	13
5.4 Indexing and retrieval	14
5.5 Prompt assembly	14
5.6 Output structure and checks	15
5.7 Chat	15
5.8 Guardrails and prompt injection	15
5.9 Evaluation	15
5.10 Interface to the backend (Proposed)	16
6. Worked example: one ELN end to end	16
7. Testing	17
8. Open decisions and risks	18

9. Implementation status and next steps	19
Appendix: glossary	19

1. Summary

MindSpark helps a relationship manager (RM) configure a structured product, see how it pays off, check it against a client profile and explain the risk in plain language. Three components do the work, and the design rule that connects them is simple: **numbers come only from deterministic code, and AI only forecasts prices and writes words.**

Component	Job	Owns the numbers?	Status
Backend (Node.js + TypeScript)	Validates input, fetches market data, calls the forecast service, computes payoff, risk and suitability, stores every simulation, calls the explanation service.	Yes. All payoff, risk and suitability figures.	Contract, client and Mode A helpers built. Rest planned.
ML forecast service (Python)	Fits GARCH(1,1) to Nifty returns, simulates 10,000 price paths for the chosen tenor and returns low, base and high paths plus a fan and sample paths.	Only price scenarios. Never payoffs.	Contract agreed. Service being built separately.
RAG service (LLM + retrieval)	Retrieves product notes and policy text and writes the plain-language explanation and chat answers from the numbers the backend sends.	No. May only repeat numbers it was given.	Proposed design (section 5).

Key ideas to remember

- Mode A does not predict a single price. It simulates thousands of possible paths and reports a range: low (5th percentile), base (median) and high (95th percentile) of the final value.
- Each case is a full daily path, so an American barrier can be checked on every day, not only at maturity.
- The payoff formulas run on those paths in the backend. The same engines serve Mode B, where the input is one shocked level.
- The backend rejects any forecast that does not match the contract. It never invents a substitute forecast.
- The explanation text is checked against the computed numbers before the RM sees it.

2. System architecture

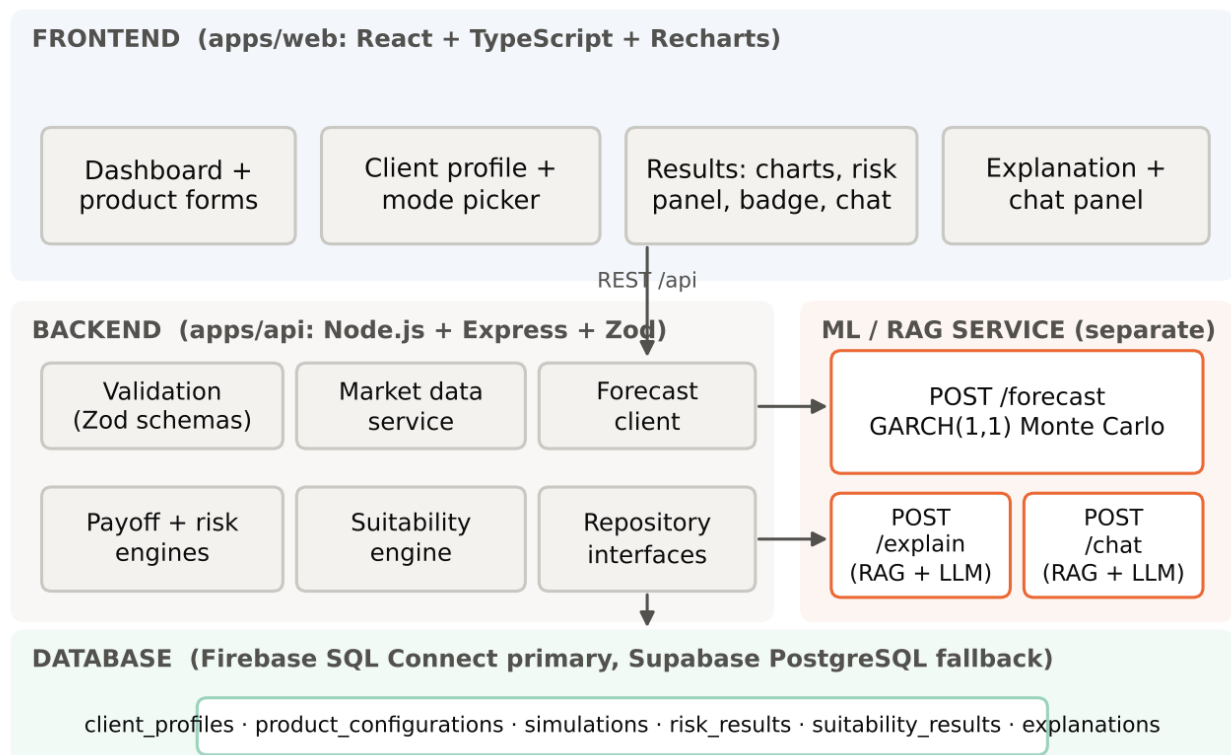


Figure 1. Components and trust boundaries. The orange area is built and operated separately; the repository only calls its endpoints.

The frontend talks only to the backend over REST under `/api`. The backend is the only component that talks to the database and to the ML/RAG service. This keeps three boundaries clean:

- **Calculation boundary.** Frontend calculations are never authoritative; the ML service never computes payoffs. Anything that affects a decision is computed in the backend engines.
- **AI boundary.** Output from the ML/RAG service is data to validate, not a source of truth. Invalid output produces an error, not a guess.
- **Database boundary.** Business logic calls repository interfaces; only adapters know about Firestore or Supabase.

2.1 Technology (Built: scaffold in repository)

Layer	Choice	Notes
Frontend	React 19, TypeScript, Vite, Recharts	Recharts draws payoff and fan charts. It only displays numbers sent by the backend.
Backend	Node.js $\geq$ 20.19, Express 5, Zod 4, TypeScript (strict)	Shared types and schemas live in packages/shared and are imported by both web and api.
Tests	Vitest, Supertest	30 unit tests exist for the forecast contract, client and Mode A helpers.
Database	Firestore SQL Connect (primary), Supabase PostgreSQL (fallback)	Failover only on connection or availability failures.
ML / RAG	Python service, owned separately	Reached through <code>AI_API_URL</code> with a bearer key held server-side.

3. Backend

3.1 Layers (Agreed)

Each request passes through the same layers, so every rule has exactly one home:

Controller	parse HTTP, call a service, map errors to status codes
Service	orchestrate one use case (simulate, suitability, explain, chat)
Engine	pure functions: payoff, risk, suitability (no I/O, fully testable)
Client	market data and forecast client (outbound HTTP, validated output)
Repository	interfaces; adapters for Firebase and Supabase behind them

Engines take plain numbers in and return plain numbers out. They never read the clock, the network or the database. That is what makes the financial logic testable against worked examples.

3.2 Endpoints (Agreed names, schemas pending)

Endpoint	Input	What the backend does	Output
POST /api/configure	Product type and terms	Validates with Zod, normalizes (for example strike as a fraction of the start level).	Normalized terms or VALIDATION_ERROR
POST /api/simulate	Terms, mode A or B, shock for B	Mode B: live level $\times (1 + \text{shock}) \rightarrow$ engines. Mode A: forecast client $\rightarrow$ case paths $\rightarrow$ engines $\rightarrow$ distribution metrics.	Payoff and risk per case, probabilities, fan data, model card
POST /api/suitability	Simulation result, client profile	Applies the rules, builds the verdict with reasons, saves the full record.	Verdict and flags
POST /api/explain	Simulation id	Sends only computed context to the explanation service; validates the reply.	Explanation text with sources
POST /api/chat	Simulation id, question	Sends context, history and question to the chat service; validates the reply.	Answer or a scope notice
GET /api/health	None	Reports status and whether the database is configured.	Status

A /api/client-profiles endpoint (save and load a profile) is needed for the PRD's "load a saved profile" and is **Proposed**, pending approval because it is not in API_SPEC.md.

3.3 Request lifecycle

Mode B (manual shock)

- Validate the request. Read the live underlying level and its timestamp from the market data service.
- Compute the shocked terminal value: $S_T = S_0 \times (1 + \text{shock})$.
- Run the product engine on the path $[S_0, S_T]$. A single point has no daily path, so American knock-in uses the terminal value (Proposed rule, shown in the UI).
- Return payoff and return in currency and percent, plus the scenario table.

Mode A (ML forecast)

- Validate the request, including the tenor range and the training window (5 or 10 years).
- Call POST {AI_API_URL}/forecast with the underlying, tenor, window and sample count. Timeout 15 s by default.
- Validate the response against the contract (section 4.9). Any failure stops here with AI_INVALID_RESPONSE or AI_UNAVAILABLE.
- Run the product engine on the low, base and high case paths. Record payoff, return and knock-in for each.
- Run the engine on every sample path to get probability of loss, probability of knock-in and payoff percentiles.
- Return results to the frontend. The 500 sample paths stay in the backend; only cases, the fan and the computed metrics are sent, because the paths can reach several megabytes.

3.4 Validation (Built: tenor and forecast rules; Proposed: product terms)

Every input is validated on the server with Zod, even though the frontend runs the same shared schemas for instant feedback. The tenor is entered in calendar days and currently limited to 30 to 1,095 days (PRD §3). This range is under review: a shorter minimum for DCD and a soft warning band for long tenors have been proposed. The limits are single constants in `packages/shared/src/constants/forecast.ts`.

Rule	Example
Positive amounts	Notional greater than zero
Barrier logic	Barrier below strike for a down-and-in ELN
Percent ranges	Protection and participation within agreed bounds (to be defined)
Tenor	Integer days within the allowed range
Training window	5 or 10 years only

3.5 Payoff engines (Proposed code, formulas Agreed)

Each product has its own pure function under `apps/api/src/engines/payoff/`. The formulas are taken verbatim from the PRD:

Product	Payoff at maturity	Notes
ELN	$N(1 + cT) - 1_{K_I} \cdot N \cdot \max(K - S_T, 0) / K$	Knock-in 1_{K_I} is checked at maturity (European) or on every daily close (American). Plain version: knocked in whenever $S_T < K$.
DCD	$N(1 + cT) \cdot \min(1, K / X_T)$	Result is in the base currency. Conversion convention still open.
CPN	$N[p + \min(\alpha \cdot \max(S_T/S_0 - 1, 0), C)]$	Cap C is optional; without a cap the min is dropped.

N is notional, c the coupon rate per year, T the tenor in years, K the strike, S_0 the starting level, S_T the level at maturity, p the protection, α the participation and C the cap. The day-count for T (proposed: `days / 365`) is an open decision.

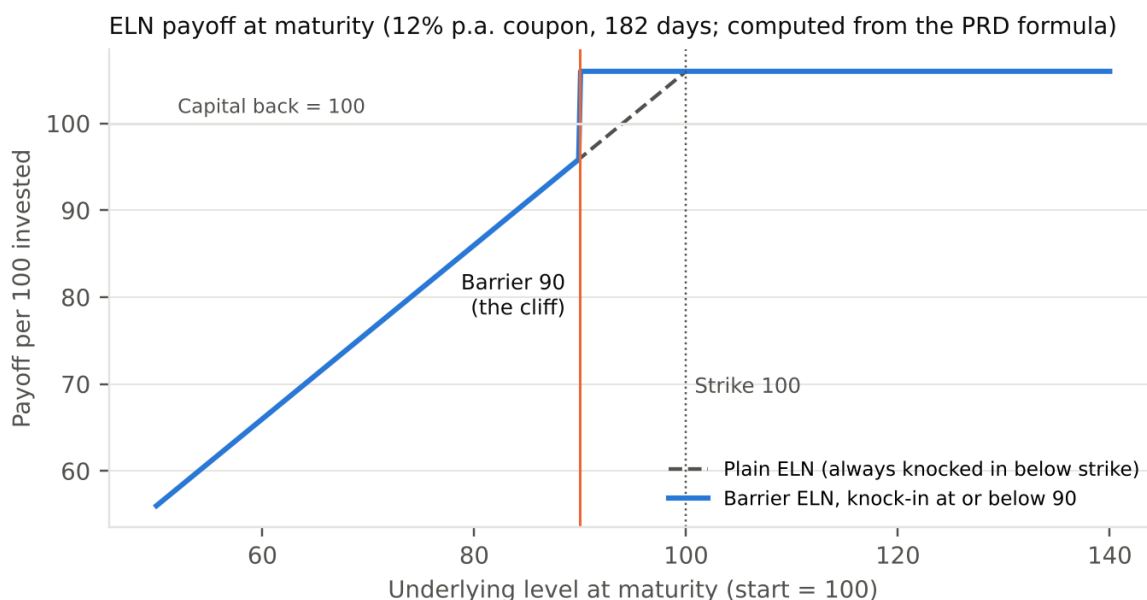


Figure 2. The ELN “cliff”: just above the barrier the investor gets the full coupon, just below it the loss from the strike applies. Computed from the PRD formula with a 90% barrier.

All engines share one signature, so Mode A and Mode B use identical code:

```

type PathPayoff = (path: readonly number[]) => number; // path[0] = S0, last = S_T

const eln: PathPayoff = (path) => {
  const knockedIn = isKnockedIn(path, barrier); // European or American
  return N * (1 + c * T) - (knockedIn ? N * Math.max(K - last(path), 0) / K : 0);
};

```

3.6 Mode A scenario processing (Built)

The file `services/simulation/modeAScenarios.ts` turns a validated forecast into engine inputs. It contains no payoff formula, so product logic stays in the engines.

Function	What it does
<code>isKnockedIn(path, barrier)</code>	European: looks only at the last value. American: checks every daily value after day 0. A flag decides whether touching the barrier counts.
<code>buildModeACases(forecast, barrier)</code>	Returns low, base and high with percentile, terminal value, path minimum and knock-in status.
<code>summarizeDistribution(forecast, payoff, invested, barrier)</code>	Runs a payoff function on all sample paths and returns probability of loss, probability of knock-in and payoff P5, P50, P95.
<code>quantileSorted(sorted, q)</code>	Linear-interpolated quantile, the same method as NumPy's default.
<code>forecastRecordMetadata(forecast)</code>	The part of the forecast stored with each simulation: model, data window, horizon, quantiles, case terminals and backtest. Sample paths and fan are dropped.

Sampling error. Probabilities are estimated from the sample paths (default 500), not all 10,000. The standard error of a probability p from n paths is $\sqrt{p(1-p)/n}$. With $p = 15\%$ and $n = 500$ that is about 1.6 percentage points. In the worked example in section 6 the American knock-in probability is 14.8% from all 10,000 paths and 16.0% from the first 500, consistent with that error. The UI should show these as approximate, and the sample size can be raised up to 2,000.

3.7 Risk engine (Proposed)

The risk engine reads engine outputs and produces the numbers shown in the risk panel and used by the suitability rules: payoff and return in each case, loss against notional, breakeven level, the scenario table (−25%, −10%, 0%, +15%) and, in Mode A, probability of loss and probability of knock-in. It is pure arithmetic over engine results.

3.8 Suitability engine (Rules Agreed as examples; mapping Proposed)

Suitability is deterministic code, never an LLM decision. The PRD lists these example rules; the engine returns a verdict (Suitable, Caution or Not suitable) with an explicit reason for every flag.

Rule	Input from simulation	Input from client profile
Low-case loss above tolerance	Loss in the low case (Mode A: 5th percentile path)	Loss tolerance
ELN knocked in in the low or base case	Knock-in status per case	None
Tenor longer than horizon	Tenor	Investment horizon
Concentration above the limit	None	Share of portfolio and the limit
Product risk above risk appetite	Product risk rating (CPN lower, ELN and DCD higher)	Risk appetite

Open decisions: which rules are “hard” (Not suitable) and which are “soft” (Caution), the allowed risk-appetite values, the concentration limit, and units for loss tolerance and horizon. A simple proposed mapping: any hard flag gives Not suitable, any other flag gives Caution, no flags gives Suitable. Whether Mode A probabilities should drive a rule or only be displayed is also undecided.

3.9 Persistence and failover (Agreed architecture)

Every simulation is stored so it can be reviewed as evidence of the suitability check: inputs, profile, results per case, verdict and flags, the explanation, and for Mode A the forecast metadata. Sample paths are not stored.

Table	Holds
client_profiles	Risk appetite, horizon, loss tolerance, concentration
product_configurations	Product type and normalized terms
simulations	Mode, shock, links to profile and configuration, forecast metadata
risk_results	One row per case: terminal value, payoff, return, knock-in, probabilities
suitability_results	Verdict and flags with reasons
explanations	Text, sources, model used

Firestore is the authoritative write target. If Firestore is confirmed unavailable (connection or availability failure only), the repository layer falls back to Supabase PostgreSQL. Validation and business errors never trigger failover, and there is no dual-write or replication. The columns above are **Proposed**; the current schema still stores each record as one JSON field.

3.10 Errors and security

Code	HTTP	When
VALIDATION_ERROR	400	Bad input, with the list of failing fields
NOT_FOUND	404	Unknown simulation or profile id
AI_UNAVAILABLE	503	ML/RAG service unreachable, timed out or returned non-2xx
AI_INVALID_RESPONSE	502	Response broke the contract; details list the issues
DATABASE_ERROR	500	Both databases failed

- Credentials (database, market data, AI key) live in environment variables and only in adapters or clients, never in controllers or engines.
- All input is validated server-side. Logs avoid secrets and unnecessary client data.
- Retrieved RAG content cannot override application instructions (SECURITY.md).

4. ML forecast service (Mode A)

This service is built and run separately. The backend depends only on the contract in section 4.9. The method below is the agreed approach and is explained here so both sides understand what the numbers mean.

4.1 What it predicts, and what it does not

An index price behaves close to a random walk, so a model that outputs one future number looks accurate in testing but does not reliably beat “tomorrow is about today.” The service therefore models the **distribution** of possible outcomes. For a tenor it returns a base case (the median) and a low and high case (the 5th and 95th percentile), delivered as daily price paths so barrier checks work. Results are scenarios, not guarantees.

4.2 Data and returns

- Input: daily closing prices of the underlying (Nifty 50, symbol $\wedge$ NSEI), 10 years by default, 5 years optional.
- The COVID period is kept. Removing a crash because it looks abnormal would make the low case too optimistic and is cherry-picking in a suitability tool.

- The model works on daily log returns $r_t = \ln(P_t / P_{t-1})$, not raw prices, because returns are roughly stationary while price levels are not.
- Trading holidays are skipped, not filled.

4.3 The GARCH(1,1) model

Volatility is not constant: calm periods and turbulent periods cluster together. GARCH captures this by letting today's variance depend on yesterday's shock and yesterday's variance:

$$\begin{aligned} r_t &= \mu + \varepsilon_t && \text{daily return} = \text{mean} + \text{shock} \\ \varepsilon_t &= \sigma_t \cdot z_t && z_t \sim \text{Student-t (standardized, } \nu \text{ degrees of freedom)} \\ \sigma_t^2 &= \omega + \alpha \cdot \varepsilon_{t-1}^2 + \beta \cdot \sigma_{t-1}^2 \end{aligned}$$

- α measures how strongly yesterday's shock raises today's variance. β measures how persistent volatility is. ω sets the long-run level.
- The long-run variance is $\omega / (1 - \alpha - \beta)$. The model needs $\alpha + \beta < 1$.
- **Student-t errors** give fat tails, so large moves such as March 2020 are more likely than under a normal distribution. That matters for the low case.
- **Mean reversion.** Expected variance h days ahead moves from today's value toward the long-run value by a factor of $(\alpha + \beta)$ per day. With $\alpha + \beta = 0.98$ the gap halves about every 34 days ($\ln 0.5 / \ln 0.98$). So short tenors reflect today's market mood, while long tenors look like the long-run average. This is the main reason very long tenors add little information.

The fit uses maximum likelihood (for example Python's arch library) on the training window.

4.4 Monte Carlo simulation

Monte Carlo means: simulate many possible futures from the fitted model and read the answer from the crowd. For each of at least 10,000 paths, the service steps forward one trading day at a time, drawing a random shock z , updating the variance with the GARCH equation, and adding the return to the log price:

```
tradingDays = round(tenorDays * 252 / 365)
for each path: var = today's variance forecast; logP = ln(spot)
  for day in 1..tradingDays:
    var =  $\omega + \alpha \cdot \varepsilon^2 + \beta \cdot \text{var}$ 
     $\varepsilon = \text{sqrt(var)} \cdot z$  # z random draw from the t distribution
    logP = logP +  $\mu + \varepsilon$ 
    price[day] = exp(logP)
```

Because the horizon is simply the number of steps, **any tenor works with the same fitted model**; there is nothing to retrain per tenor. A longer tenor means more steps and a wider spread of outcomes, roughly growing with the square root of time. The tenor limits (currently 30 to 1,095 days) are product and validation choices, not limits of the method.

4.5 Cases, fan and samples

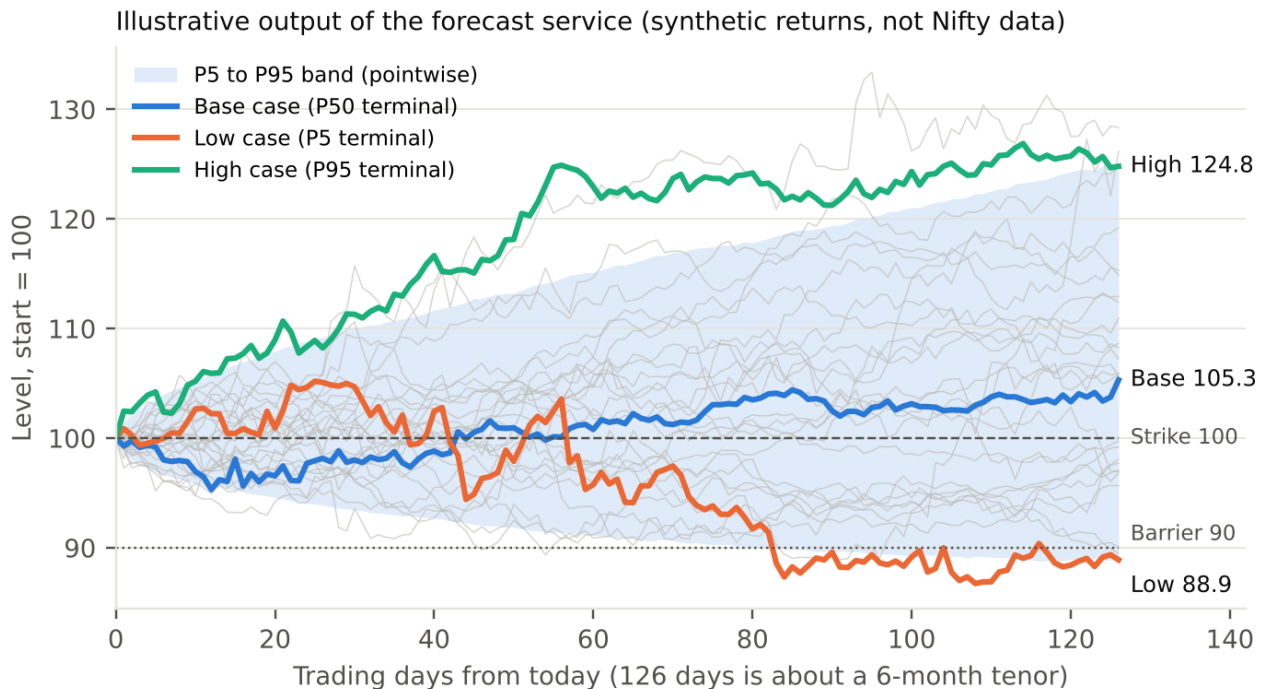


Figure 3. What the service returns for a 6-month tenor. Synthetic returns generated for illustration with $\omega = 2e-6$, $\alpha = 0.09$, $\beta = 0.89$, $\nu = 6$; this is not Nifty data.

Output	Definition	Used for
Low, base, high case	The simulated path whose final value is closest to the 5th, 50th and 95th percentile of all final values	Payoff, return and knock-in per case; suitability low case
Fan (P5, P50, P95)	Percentiles of the simulated level at each day	The shaded band on the forecast chart
Sample paths	A random subset of full paths (default 500, up to 2,000)	Probability of loss and of knock-in, payoff percentiles
Terminal quantiles	P5, P50 and P95 of the final value	Headline range: "Base X, likely range L to H"

A case path is a real simulated path, not a line drawn through percentiles. Its intermediate days can sit outside the pointwise fan band; that is expected. Using real paths is what allows an American barrier to be checked honestly.

4.6 Drift (average return)

Over a short tenor the average return barely matters. Over several years it noticeably shifts the base case. The contract returns which method was used. Options are the historical mean of the training window, or a fixed, documented assumption (for example 10 to 12% per year). A strong rally in the window, such as 2020 to 2021, can inflate the historical mean, so the choice should be stated in the model card. Volatility, not drift, drives the width of the low and high bands.

4.7 Training window

- **10 years (default).** It includes real crashes, gives a more stable average, and gives GARCH enough data to learn how volatility spikes and settles.
- **5 years (option).** More weight on recent conditions but misses the March 2020 fall of about 38% in about a month, so the low case is less cautious.
- Single stocks that changed a lot (merger, demerger) may need 3 to 5 years. Indices and FX pairs are fine with 10.
- Choose between windows with the backtest, not by taste.

4.8 Backtest: how forecast error is measured

The PRD asks the UI to show the model's past forecast error. The service reports a walk-forward backtest at the same horizon as the request:

- Pick a past origin date. Fit on data before it. Simulate the horizon. Compare the real value at the end of the horizon with the forecast.
- **Band coverage:** how often the real value fell between low (P5) and high (P95). For a 90% band the target is about 90%.
- **Base MAPE** and **naïve MAPE:** average percentage error of the median forecast and of a “no change” forecast. A model is only useful if it is not worse than naïve on the point forecast, and the band is what adds value.
- Move the origin forward and repeat. Never split data randomly; always by date, so future information cannot leak into training.

Statistical caveat. Overlapping windows are strongly correlated, so they do not count as independent tests. With 10 years of data and a 126-day horizon there are only about 16 to 20 independent windows. The standard error of a 90% coverage estimate from 16 to 20 windows is roughly 6 to 8 percentage points. The PRD target of 80 to 95% coverage is therefore a sanity check, not a precise proof, and the UI should say how many windows were used. At a 3-year horizon there are only about 3 independent windows, so coverage cannot be verified. This is the main reason for a cap on tenor.

4.9 Contract with the backend (Agreed)

Endpoint: POST {AI_API_URL}/forecast, bearer-token authentication. Full details are in docs/forecasting.md. Request fields: underlying symbol and asset class, tenorDays, trainingWindowYears (5 or 10), samplePathCount (100 to 2,000). The response contains the model description, data window, horizon, three case paths, terminal quantiles, fan, sample paths and backtest.

The backend (Built) rejects the response unless all of these hold:

- contractVersion is 1.0, and the model ran at least 10,000 simulations.
- Every case path, fan series and sample path has exactly tradingDays + 1 values and starts at the reported spot.
- All prices are finite and positive; the horizon and symbol match the request; the sample count matches.
- $P5 \leq P50 \leq P95$, and the final values of the cases satisfy $low \leq base \leq high$.
- The data is no more than 5 calendar days old, and training start is before training end.
- Backtest coverage is between 0 and 1 and the errors are not negative.

Failures return AI_INVALID_RESPONSE with the list of issues; network errors, timeouts and non-2xx responses return AI_UNAVAILABLE. Payload size for a 3-year tenor with 500 sample paths is roughly 3 to 4 MB of JSON.

4.10 Optional ML variant

A learned model can be added alongside GARCH: LightGBM quantile regression with quantiles 0.05, 0.5 and 0.95, predicting the return over the tenor from lagged returns (5, 20, 60 days), realized volatility, distance from the 50 and 200 day averages and India VIX, split by date. It does not produce paths, so it can supply low, base and high end values but not barrier checks. Keep the GARCH Monte Carlo as the baseline and show both in the backtest; on an index the learned model rarely beats GARCH by much, and saying so makes the project more credible.

4.11 Limitations to state in the product

- It is a statistical scenario model, not a price prediction or an issuer fair-value calculation.
- It assumes the future resembles the training window; regime changes and events outside it are not captured.

- GARCH(1,1) does not model jumps explicitly, relationships between assets, or macro events.
- Bands become mostly assumption at long tenors (volatility has reverted, drift dominates).
- Forecast quality is measured on few independent windows, so coverage numbers are approximate.

5. RAG: retrieval-augmented explanation and chat

Status: Proposed design. The PRD defines what the RAG layer must do (explain, answer follow-ups, obey guardrails). The repository does not yet specify how it is built. This section is a recommended design for the separately developed service; choices such as the embedding model and vector store are open.

5.1 Role

RAG (retrieval-augmented generation) lets a language model answer from a curated set of documents instead of from memory. For MindSpark it does two things: write a plain-language explanation of a simulation, and answer the RM's follow-up questions. It **explains only**. Every number comes from the backend; the model's job is to put those numbers into words a non-expert client can follow, using the right product and policy language.

5.2 What the explanation must cover (PRD §7.3)

- What the product is.
- The best case.
- The worst case.
- What triggers a loss.
- Why it is or is not suitable for this client.
- Risk wording, always present, with jargon defined simply.

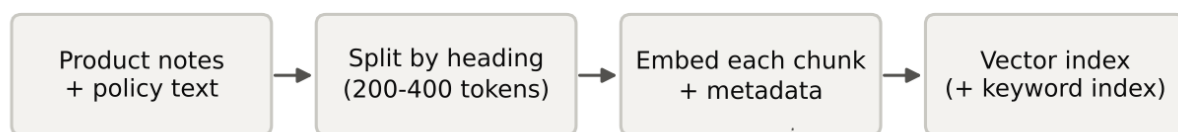
5.3 Corpus

Document set	Contents	Metadata kept on every chunk
Product notes	ELN, DCD and CPN descriptions, how the payoff works, key risks, who the product is for, glossary	product, section, document version
Suitability policy text	What each rule means and why it exists (horizon mismatch, concentration, risk appetite, loss tolerance)	rule id, version
Wording guide	Approved phrases for risk notices and how to define jargon	type = style

The corpus is small and internal, written and reviewed by the team. It is not scraped content, which keeps the retrieval problem simple and the injection risk low.

5.4 Indexing and retrieval

OFFLINE: build the index



ONLINE: answer one request

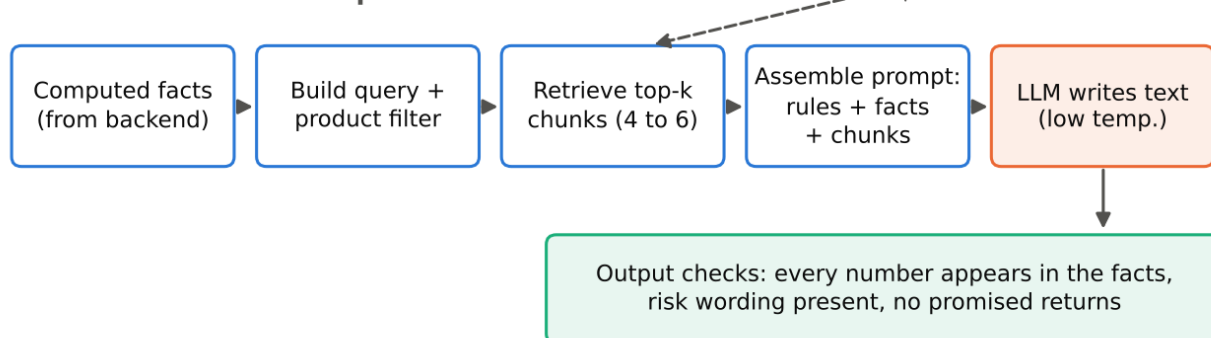


Figure 4. Offline indexing and the online path for one explanation or chat request.

- **Chunking.** Split by heading into passages of roughly 200 to 400 tokens, so each chunk is one idea (for example “ELN: key risks”). Small overlap only inside long sections.
- **Embedding.** Convert each chunk to a vector with a sentence-embedding model and store the vector with its metadata in the ML service's own index (for example FAISS or a small vector store). This is separate from the application database.
- **Hybrid search.** Combine vector similarity with keyword (BM25) search and merge the rankings. Keywords help with exact terms such as “knock-in” or “barrier.”
- **Metadata filter.** Search only the chunks for the product in question plus the policy chunks, so an ELN explanation never retrieves CPN text.
- **Top-k.** Return the best 4 to 6 chunks above a similarity threshold. If none passes, the answer must say the topic is not covered rather than guess.

How the query is built

For **/explain** the query is deterministic, not free text: one query per explanation section (for example “ELN worst case knock-in loss”), plus one policy query for each flag the suitability engine raised. A flag for tenor longer than horizon pulls the horizon-mismatch policy chunk automatically. For **/chat** the query is the RM's question plus the product, with the last few turns for context.

5.5 Prompt assembly

The prompt has four clearly separated parts. Order and labelling matter because they set which text is allowed to instruct the model:

Part	Content	Treated as
1. Rules	Role, tone for a non-expert, the six required sections, banned claims (no promised returns, no advice beyond the verdict), always include the risk notice, use only numbers in the facts	Instructions
2. Facts	JSON from the backend: product terms, results per case, probabilities, verdict and flags with reasons, client profile summary, model card (training window, backtest)	Authoritative data
3. Passages	The retrieved chunks, each labelled with its source	Reference text. Cannot change the rules
4. Task	Produce the explanation in the required JSON structure (or answer this question)	Instructions

Generation uses a low temperature (0 to 0.2) so the same facts give nearly the same text. Mode A prompts require range wording: “base about X, likely range L to H,” never a single predicted value.

5.6 Output structure and checks

```
{ "what_it_is": "...", "best_case": "...", "worst_case": "...",  
  "loss_triggers": "...", "suitability_reasoning": "...", "risk_notice": "...",  
  "sources": ["eln-notes#key-risks", "policy#horizon-mismatch"] }
```

The reply is accepted only if it passes automatic checks, first in the ML service and again in the backend:

- **Number check.** Extract every number and percentage from the text, normalize the format and confirm each appears in the facts (allowing for rounding). Any unknown number rejects the reply.
- **Banned phrases.** “Guaranteed,” “risk-free,” “will earn,” and similar.
- **Required content.** Risk notice present; all six sections present; Mode A text includes a range.
- **On failure.** Regenerate once with a stricter instruction. If it still fails, return AI_INVALID_RESPONSE. The backend can then show the computed numbers and flags without prose; it does not show an unchecked explanation.

5.7 Chat

- **Grounding.** Each answer uses the current simulation's facts and the retrieved passages. History is limited to the last few turns to control cost and drift.
- **Scope check.** Questions about this simulation, the product and the suitability result are in scope. Market tips, other products and personal investment advice are not; they get a short, polite scope notice.
- **No new calculations.** If the RM asks “what if the barrier were 85%,” the chat explains that the simulation must be re-run with the new term. It never computes the result itself.
- **Not covered.** If retrieval finds nothing relevant, say so.

5.8 Guardrails and prompt injection

- Instruction hierarchy: rules and facts outrank passages and user text. Passages and RM questions are data, so “ignore previous instructions” inside them has no authority (SECURITY.md).
- The corpus is curated internally and reviewed before indexing; nothing from users or the web is added automatically.
- The model has no tools: it cannot call the database, change a result or place an order.
- No promised returns, no advice beyond the computed verdict, risk wording always present, jargon defined in simple words.
- The backend, not the model, is the authority for the verdict. A reply that contradicts the verdict is rejected.

5.9 Evaluation

What	How	Pass condition
Retrieval quality	30 to 50 questions with the expected chunks marked; measure recall in the top 5	Right chunk in the top 5 for almost all questions
Numeric faithfulness	Automated number check over a fixed set of simulations	Zero unexplained numbers
Wording safety	Banned-phrase and risk-notice checks; adversarial prompts	No violations
Readability	RM and a non-expert reviewer read a sample	Understandable and consistent with the numbers
AI boundary (in repo)	Integration test where the AI reply contains different numbers	Stored payoff, risk and verdict are unchanged

5.10 Interface to the backend (Proposed)

Both endpoints use the same bearer authentication as /forecast. The request carries only computed context; the reply carries text and sources.

```
POST /explain { product, profile, results, suitability: {verdict, flags}, forecast }
               → { text | sections, sources: [...], model }
POST /chat   { context, history: [{role, text}], question }
               → { text, inScope: boolean, sources: [...] }
```

6. Worked example: one ELN end to end

Illustrative only. All market numbers come from the synthetic simulation behind Figure 3, not from Nifty data. The client profile and rule thresholds are invented to show the flow; real values depend on decisions listed in section 8.

6.1 Inputs

Item	Value
Product	Barrier ELN, American barrier, down-and-in
Notional N	₹10,00,000
Coupon c	12% per year
Tenor	182 days ($T = 182/365 = 0.4986$ years), 126 trading days
Strike K / barrier	100% / 90% of the starting level
Coupon amount $N \cdot c \cdot T$	₹59,836 (5.98% of N)
Client	Loss tolerance 5% of notional; horizon 12 months; moderate risk appetite (illustrative)

6.2 Forecast response (Mode A)

The forecast service returns low, base and high paths ending at 88.9, 105.3 and 124.8 (start = 100), plus a fan and 500 sample paths. The backend validates the response, then runs the ELN engine on each path.

Case	Final level	Lowest daily level	Knocked in? (barrier 90)	Payoff	Return
Low (P5)	88.9	86.7	Yes	₹9,48,795	-5.12%
Base (P50)	105.3	95.3	No	₹10,59,836	+5.98%
High (P95)	124.8	102.3	No	₹10,59,836	+5.98%

Low-case calculation. The low path touched 86.7, below the 90 barrier, so the ELN is knocked in. The loss from the strike is $N \times (100 - 88.9) / 100 = ₹1,11,040$. Payoff = ₹10,59,836 – ₹1,11,040 = **₹9,48,795**, a loss of 5.12% of the notional even though the coupon was paid.

6.3 Distribution metrics from the sample paths

Metric	All 10,000 paths	First 500 (sample sent to backend)
Probability the index ends below its start	29.5%	not reported
Probability of knock-in, American barrier	14.8%	16.0%
Probability of knock-in, European barrier	6.3%	7.4%
Probability the ELN returns less than the notional	9.8%	10.2%
ELN payoff P5 / P50 / P95	₹9,48,794 / ₹10,59,836 / ₹10,59,836	similar

Two lessons from the numbers. First, the American barrier is knocked in more than twice as often as the European one (14.8% vs 6.3%): a path can dip below the barrier and recover. A model that only produced end prices would understate the risk. Second, the index ends below its start in 30% of paths, but the ELN loses money in only about 10%, because the loss needs a knock-in as well as a fall below the strike. The coupon cushions the rest.

6.4 Suitability (illustrative rules)

Rule	Result	Flag
Low-case loss above tolerance	Loss 5.12% against tolerance 5%	Raised
ELN knocked in in the low case	Yes	Raised
Tenor longer than horizon	6 months against 12 months	Not raised
Concentration above limit	Within limit (illustrative)	Not raised
Product risk above appetite	ELN rated high, client moderate	Raised

With the proposed mapping and the first rule treated as hard, the verdict would be **Not suitable**, with three listed reasons. If no rule were hard, three soft flags would give Caution. The thresholds, and which rules are hard, are decisions for the product owner.

6.5 Explanation (what the RAG layer might write)

“This note pays a fixed 12% a year as long as the index does not fall to 90% of its starting level at any point during the six months. In the low scenario the index falls below that level, the protection stops, and you take the index loss: about ₹9,48,795 back on ₹10,00,000 invested. In the base scenario you receive about ₹10,59,836. The low and high scenarios are examples from a statistical model, not predictions. For this client the loss in the low scenario exceeds the 5% they said they can accept, the barrier is touched in that scenario, and the product's risk level is above their stated appetite. This is a simulation, not a guarantee.”

Every number in the sample text appears in the facts the backend sent, so it would pass the number check. The checks in section 5.6 run before the RM sees it.

7. Testing

Level	What is tested	Status
Unit: forecast contract	Request limits, tenor to trading days, response validation (lengths, spot anchor, positivity, ordering, staleness, version)	Built, passing
Unit: forecast client	Request normalization, error mapping to VALIDATION_ERROR, AI_UNAVAILABLE, AI_INVALID_RESPONSE with a stubbed fetch	Built, passing
Unit: Mode A helpers	Knock-in on paths (American vs European, barrier touch, day 0), case building, distribution metrics, quantiles, stored metadata	Built, passing
Unit: payoff engines	ELN, DCD, CPN against the worked examples in the product notes	Planned; examples still needed
Unit: risk and suitability	Each rule, flag combinations, verdict mapping	Planned
Integration	configure, simulate (A and B), suitability, AI boundary, persistence and failover	Planned
Contract test	A recorded real forecast response must pass validateForecastResponse	Planned; needs the ML service
AI boundary	An AI reply with different numbers must not change stored results	Planned
End to end	Dashboard → product → profile → mode → results → suitability → explanation → chat	Planned

Test fixtures for forecasts are built deterministically in `tests/fixtures/forecast.ts` and are used only in tests. Production code never substitutes fake market data.

8. Open decisions and risks

8.1 Decisions needed before payoff code

Question	Suggested answer
Coupon year fraction T	<code>tenorDays / 365</code>
Strike and barrier	Percent of the starting level
Does touching the barrier count as knock-in?	Yes
DCD base currency and quote convention	Deposit currency; convention to confirm
American barrier in Mode B (no daily path)	Knock-in if the shocked value breaches the barrier; say so in the UI
Verdict mapping and hard versus soft rules	Any hard flag gives Not suitable; any other flag gives Caution
Product risk ratings	CPN low, DCD high, ELN high
Saved client profiles endpoint	Add <code>/api/client-profiles</code>
Tenor range	Per-product ranges or a soft limit with a warning band (under discussion)
Worked payoff examples from the product notes	Required as tests

8.2 Risks

Risk	Why it matters	Mitigation
Forecast looks more precise than it is	RM or client may read the base case as a prediction	Always show a range, model card and backtest; wording rules in the explanation
Few independent backtest windows	Coverage estimate is noisy, long tenors cannot be validated	Report window count; cap or warn on long tenors
Drift assumption dominates long tenors	Base case reflects the assumption, not evidence	State the drift method; use volatility for risk bands
Stale or inconsistent market data	Spot in forecast and live level could differ	Reject forecasts older than 5 days; use one provider if possible
LLM states a wrong number	Mis-selling risk and loss of trust	Number whitelist check in service and backend; deterministic fallback
Large forecast payload	3 to 4 MB per long-tenor response	Keep sample paths in the backend; decimate the fan for the browser
Another developer owns the ML side	Contract drift breaks Mode A	Version the contract; shared schema; contract test with a recorded response

9. Implementation status and next steps

Area	Done	Next
Requirements	PRD v2.0, forecast contract, implementation plan, updated supporting docs	Settle the decisions in 8.1
Shared package	Forecast constants and Zod schemas, tenor conversion, new error codes	Product and profile schemas
Backend	Forecast client, Mode A scenario helpers, 30 unit tests	Server, configure, payoff and risk engines, Mode B, then Mode A endpoint, suitability, persistence
ML forecast	Contract and method agreed	Build /forecast, share one real Nifty response for the contract test
RAG	Requirements and a proposed design	Choose embedding model and store, write the corpus, build /explain and /chat
Frontend	Scaffold, Recharts chosen	Forms, results page, fan chart, model card
Database	Architecture and logical tables	Typed schema, migration, adapters, failover

Appendix: glossary

Term	Meaning
ELN	Equity-linked note (reverse convertible): high coupon, the client bears downside on the underlying.
DCD	Dual currency deposit: short deposit with a high rate, may be repaid in the other currency.
CPN	Capital-protected note: protected floor plus partial upside.
Knock-in	The event where a barrier is breached and downside protection stops.
European / American barrier	Checked only at maturity / checked on every daily close.
Log return	$\ln(\text{today's price} / \text{yesterday's price})$.
GARCH	A volatility model where variance depends on recent shocks and recent variance.
Monte Carlo	Estimating outcomes by simulating many random scenarios.
Percentile (P5, P50, P95)	The value below which 5%, 50% or 95% of simulated outcomes fall.
Band coverage	Share of past real outcomes that fell inside the predicted low-to-high band.
MAPE	Mean absolute percentage error.
RAG	Retrieval-augmented generation: a language model answering from retrieved documents.
Embedding	A list of numbers representing the meaning of a text, used to find similar passages.